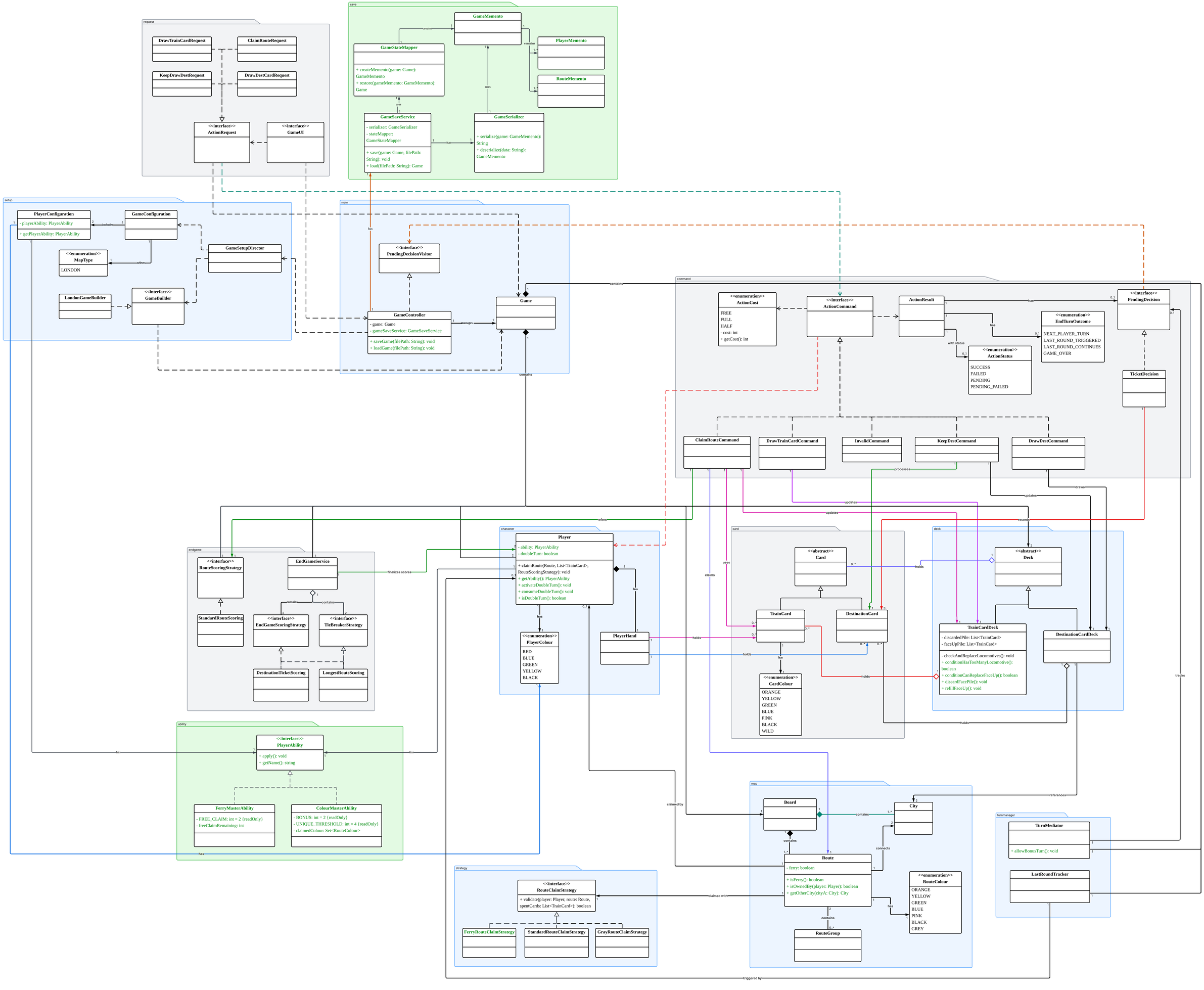

request
DrawTrainCardRequest
ClaimRouteRequest
KeepDrawDestRequest
DrawDestCardRequest
<<interface>>
ActionRequest
<<interface>>
GameUI
save
GameMemento
creates
GameStateMapper
+ createMemento(game: Game): GameMemento
+ restore(gameMemento: GameMemento): Game
contains
PlayerMemento
RouteMemento
uses
GameSaveService
- serializer: GameSerializer
- stateMapper: GameStateMapper
+ save(game: Game, filePath: String): void
+ load(filePath: String): Game
has
GameSerializer
+ serialize(game: GameMemento): String
+ deserialize(data: String): GameMemento
setup
PlayerConfiguration
- playerAbility: PlayerAbility
+ getPlayerAbility: PlayerAbility
includes
GameConfiguration
<<enumeration>>
MapType
LONDON
selects
GameSetupDirector
LondonGameBuilder
<<interface>>
GameBuilder
main
<<interface>>
PendingDecisionVisitor
GameController
- game: Game
- gameSaveService: GameSaveService
+ saveGame(filePath: String): void
+ loadGame(filePath: String): void
manages
Game
contains
command
<<enumeration>>
ActionCost
FREE
FULL
HALF
- cost: int
+ getCost(): int
<<interface>>
ActionCommand
ActionResult
has
<<interface>>
PendingDecision
<<enumeration>>
EndTurnOutcome
NEXT_PLAYER_TURN
LAST_ROUND_TRIGGERED
LAST_ROUND_CONTINUES
GAME_OVER
with status
<<enumeration>>
ActionStatus
SUCCESS
FAILED
PENDING
PENDING_FAILED
TicketDecision
ClaimRouteCommand
DrawTrainCardCommand
InvalidCommand
KeepDestCommand
DrawDestCommand
processes
draws
updates
records
refers
endgame
<<interface>>
RouteScoringStrategy
EndGameService
finalizes scores
StandardRouteScoring
<<interface>>
EndGameScoringStrategy
<<interface>>
TieBreakerStrategy
DestinationTicketScoring
LongestRouteScoring
character
Player
- ability: PlayerAbility
- doubleTurn: boolean
+ claimRoute(Route, List<TrainCard>, RouteScoringStrategy): void
+ getAbility(): PlayerAbility
+ activateDoubleTurn(): void
+ consumeDoubleTurn(): void
+ isDoubleTurn(): boolean
<<enumeration>>
PlayerColour
RED
BLUE
GREEN
YELLOW
BLACK
PlayerHand
holds
card
<<abstract>>
Card
TrainCard
DestinationCard
<<enumeration>>
CardColour
ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
WILD
deck
<<abstract>>
Deck
TrainCardDeck
- discardedPile: List<TrainCard>
- faceUpPile: List<TrainCard>
- checkAndReplaceLocomotives(): void
+ conditionHasTooManyLocomotive(): boolean
+ conditionCanReplaceFaceUp(): boolean
+ discardFacePile(): void
+ refillFaceUp(): void
DestinationCardDeck
tracks
ability
<<interface>>
PlayerAbility
+ apply(): void
+ getName(): string
FerryMasterAbility
- FREE_CLAIM: int = 2 {readOnly}
- freeClaimRemaining: int
ColourMasterAbility
- BONUS: int = 2 {readOnly}
- UNIQUE_THRESHOLD: int = 4 {readOnly}
- claimedColour: Set<RouteColour>
map
Board
contains
City
Route
- ferry: boolean
+ isFerry(): boolean
+ isOwnedBy(player: Player): boolean
+ getOtherCity(cityA: City): City
connects
<<enumeration>>
RouteColour
ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
GREY
RouteGroup
claimed by
claimed with
strategy
<<interface>>
RouteClaimStrategy
+ validate(player: Player, route: Route, spentCards: List<TrainCard>): boolean
FerryRouteClaimStrategy
StandardRouteClaimStrategy
GrayRouteClaimStrategy
turnmanager
TurnMediator
+ allowBonusTurn(): void
LastRoundTracker
references
triggered by